

Luxair interjú Q&A gyűjtemény

Kérdések és válaszok külön PDF-ben

100 kérdés-válasz, 10 témakörből

Az eredeti PDF-ek Q&A részei alapján összegyűjtve. Az angol válaszokat megtartottam, hogy interjúra közvetlenül gyakorolható legyen.

Tartalom

1. API design at scale
2. API security
3. Azure, Docker, CI/CD, deployment and testing
4. Data consistency and distributed systems
5. Database topics
6. Integration patterns and Mule to Node.js migration
7. Node.js, Express and TypeScript
8. Performance and scaling
9. Production debugging and observability
10. SOA, microservices, monolith and service boundaries

1. API design at scale

1. Hogyan terveznel egy public API-t, amit több partner is használ?

Answer: Eloszor contract-first gondolkodnék, például OpenAPI spec-kel. Stabil endpoint naming, helyes status codeok, strukturált error format, auth, rate limiting és versioning policy kell. Public API-nál a backward compatibility kritikus, ezért alából additive change-eket csinálnék, breaking change esetén pedig új versiont, deprecation window-t és usage monitoringot használnék.

2. Mi a különbség public és private API között?

Answer: Public API-nál külső partnerek vagy széles consumer kör függ az interface-től, ezért dokumentáció, versioning, security, rate limit és deprecation policy nagyon fontos. Private API belső használatú, de ha sok internal app függ tőle, ott is úgy kell kezelni, mint egy contractot. A lényeg nem csak az, hogy internetről elérhető-e, hanem hogy hány consumerre van hatással egy változás.

3. Hogyan versionálnál egy API-t breaking change esetén?

Answer: Alából elkerülném a breaking change-et és additív változásokat preferálnék. Ha mégis kell breaking change, létrehoznék egy /v2 endpointot a /v1 mellett, párhuzamosan futtatnám őket, mert nem a /v1 usage-et, kommunikálnám a deprecation timeline-t, és csak akkor kapcsolnám ki a régit, ha a consumerok átálltak vagy van velük megállapodás.

4. Milyen API változás breaking és milyen nem breaking?

Answer: Új optional field vagy új endpoint általában non-breaking. Field törles, field típusváltás, required request field hozzáadása, vagy egy enum érték jelentésének megváltoztatása breaking lehet. Interjúban azt mondanám, hogy additive-only by default, breaking change csak versioninggel vagy egyeztetett migration window-val.

5. Hogyan paginalnál egy nagy flight search response-t?

Answer: Nem adnám vissza egyben az összes találatot. Kis datasetnél offset/limit egyszerű lehet, de flight searchnél inkább cursor vagy keyset paginationt használnék stabil rendezési kulccsal, például departure_time plusz flight_id. Így a következő oldal nem csuszik el, ha közben adatok változnak, és a deep pagination sem lesz olyan drága. Tényleg max page size limitet is.

6. A consumer elküldi a POST /bookings requestet, timeoutot kap, majd retryol. Mit kell tenni?

Answer: Ez klasszikus idempotency probléma. Lehet, hogy az első request szerveroldalon sikerült, csak a response veszett el. Ezért a client küldjön Idempotency-Key headert. A szerver az első feldolgozásnál eltárolja a key-t és az eredményt. Ha ugyanaz a key újra jön, nem hoz létre második bookingot, hanem visszaadja az eredeti eredményt vagy kezeli a pending állapotot.

7. Milyen legyen egy jó API error response?

Answer: A status code mellett legyen strukturált body. Például machine-readable code, human-readable message, request_id vagy correlation_id, és details validációs hibákhoz. A code stabil, hogy client program tudjon rá reagálni, a message embernek segít, a request_id pedig operations oldalon segít megtalálni a logokat és trace-eket. Stack trace-t vagy belső secretet nem adnék vissza.

8. Mi a különbség 401 és 403 között?

Answer: 401 azt jelenti, hogy a request nincs rendesen autentikálva, például hiányzik, lejárt vagy invalid a token. 403 azt jelenti, hogy tudjuk, ki vagy, de nincs jogosultságod ehhez az actionhoz vagy resource-hoz. Egyszerűen: 401 = nem tudom, ki vagy; 403 = tudom, ki vagy, de nem teheted meg.

9. Miert fontos a correlation ID production debuggingban?

Answer: Mert egy request több service-en, queue-n és downstream hívásokon mehet át. Ha minden logban, outbound headerben és message metadatan ott van ugyanaz a correlation ID, akkor incidentnél össze lehet kötni az egész utvonalat. Ez különösen fontos integration-heavy rendszerekben, ahol a hiba lehet nálunk, egy belső service-nél vagy egy third-party partnernél.

10. Mikor választanál REST-et, GraphQL-t vagy gRPC-t?

Answer: Public vagy partner API-hoz általában REST-et választanék, mert egyszerű, HTTP-native, jól dokumentálható és széles körben ismert. GraphQL akkor jó, ha egy kliensnek rugalmasan kell sok backendből adatot összegyűjtenie. gRPC inkább

internal service-to-service kommunikációhoz éros, ahol performance, schema és code generation fontos. Nem választanék GraphQL-t vagy gRPC-t csak azért, mert modernebbnek hangzik.

2. API security

1. What is the difference between authentication and authorization?

Answer: Authentication answers who the caller is. Authorization answers what that caller is allowed to do. In an API I usually treat them as separate layers: first validate the token or credential, then check scopes, roles, ownership, or business policy. A user can be authenticated but still forbidden from accessing a specific resource.

2. What is the difference between 401 and 403?

Answer: 401 means the request is not properly authenticated, for example the token is missing, invalid, or expired. 403 means the caller is authenticated, but does not have permission for this action or resource. In some sensitive cases we may return 404 instead of 403 to avoid revealing that a resource exists, but the underlying problem is still authorization.

3. Walk me through service-to-service authentication with OAuth 2.0.

Answer: For machine-to-machine calls I would normally use the Client Credentials flow. Service A is registered with the identity provider and requests an access token for Service B's audience or scope. Service A sends that token as a Bearer token. Service B validates the signature, issuer, audience, expiry, and required permissions locally, usually using cached public keys from the issuer.

4. What should you validate in a JWT?

Answer: Parsing a JWT is not enough. I need to verify the signature using the issuer's public key, and then validate important claims such as issuer, audience, expiration, not-before, and scopes or roles. I would also configure the allowed algorithms explicitly and avoid trusting the token header to choose the verification mode.

5. Why should you not put secrets into a JWT payload?

Answer: A normal JWT is signed, not encrypted. The signature protects integrity, but anyone who has the token can still read the payload. So I would not put passwords, API keys, private data, or internal secrets into token claims. Tokens should contain only the minimum claims needed for authorization decisions.

6. JWT or opaque token, which one would you choose?

Answer: It depends on the use case. JWTs are self-contained and can be validated locally, so they are good for high-volume service-to-service APIs with short expiries. Opaque tokens require introspection at the authorization server, which adds latency but gives better immediate revocation. For internal high-throughput APIs I would often prefer short-lived JWTs. For user sessions where immediate logout matters, opaque or hybrid can make sense.

7. What is BOLA and how do you prevent it?

Answer: BOLA means Broken Object Level Authorization. The API checks that the caller is logged in, but forgets to check whether the caller owns or may access the specific object. Example: user A can call GET /bookings/123 and get user B's booking. I would prevent it with resource-level authorization on every data access path, policy checks close to the data access, and tests where one user tries to access another user's data.

8. How would you secure a webhook endpoint?

Answer: I would validate the provider's signature, usually with HMAC or certificate-based validation, and include timestamp or nonce checks to reduce replay risk. I would validate the payload schema, process events idempotently using an event id, log with a correlation id, and avoid doing long work directly in the request. Ideally I acknowledge quickly and process through a queue or worker.

9. How do you handle secrets in a containerized cloud application?

Answer: I would not put secrets in source code, Docker images, or plain config files. In Azure I would use Key Vault or an equivalent secret manager, accessed through managed identity where possible. Secrets should be injected at runtime, rotated regularly, and redacted from logs. For local development, .env files are acceptable only if they are gitignored and clearly separated from production.

10. How do you design safe error responses for public APIs?

Answer: The client should receive a consistent error shape with a stable error code, a clear message, and a request id for support. I would avoid exposing stack traces, SQL errors, internal hostnames, token validation internals, or secrets. Detailed diagnostics go to structured logs and traces, with sensitive values redacted. This gives consumers useful information without leaking internal implementation details.

3. Azure, Docker, CI/CD, deployment and testing

1. Which Azure services would you choose for a Node-based integration platform?

Answer: I would start with APIM at the edge for external-facing APIs, Service Bus for durable asynchronous messaging, Key Vault plus Managed Identity for secrets, and App Service or Container Apps for long-running Node services. I would use Functions for small trigger-driven tasks and Event Grid when the requirement is notification-style events rather than durable business work.

2. When would you use Azure Service Bus instead of Event Grid?

Answer: I would use Service Bus when I need reliable business messaging, queues or topics, retry handling, dead-letter queues, duplicate detection, or controlled consumer processing. Event Grid is better for lightweight push-style event notifications. They solve different reliability and delivery problems.

3. When would you use Azure Functions for integration work?

Answer: Functions are a good fit for small event-driven tasks such as webhook receivers, timers, blob triggers, or simple transformations. I would be careful with complex core integration flows because a long-running Node service is often easier to test, observe, debug, and operate.

4. How would you secure secrets for a Node service running in Azure?

Answer: I would avoid storing secrets in code, Git, or Docker images. In Azure I would use Key Vault and let the service access it through Managed Identity. That way the application authenticates as itself, permissions are controlled by the platform, and secrets can be rotated without rebuilding the application.

5. What is the benefit of Docker in this role?

Answer: Docker packages the application and its runtime in a repeatable way, which reduces environment differences and fits well with CI/CD. For a Node service I would care about a clean Dockerfile, small image size, environment-based config, stdout logging, health checks, and not running the process as root in production.

6. What would a good CI/CD pipeline for a Node integration service include?

Answer: A good pipeline would install dependencies using a lockfile, compile TypeScript, run linting, unit tests, integration tests, and contract tests where relevant. It would also run security scans, build a Docker image tagged with the commit, deploy to staging, run smoke or E2E checks, and then promote to production with a controlled rollout and rollback option.

7. What is the difference between rolling, blue-green, and canary deployment?

Answer: Rolling replaces instances gradually. Blue-green keeps two full environments and switches traffic, which gives fast rollback but costs more. Canary sends a small percentage of traffic to the new version first and increases it only if metrics look healthy. For high-risk or high-traffic services, canary is often the safest if observability is strong enough.

8. How would you deploy a risky change safely?

Answer: I would first make sure the change is covered by tests and ideally guarded by a feature flag. Then I would deploy to staging, run smoke tests, and roll out gradually with canary or blue-green depending on the platform. During the rollout I would watch latency, error rate, dependency failures, and business metrics, and I would define rollback criteria before starting.

9. What testing layers matter for an integration service?

Answer: Unit tests are useful for isolated logic, but integration services need more. I would add integration tests for database or messaging boundaries, contract tests for API compatibility, and smoke or E2E tests after deployment. I would also test failure cases such as validation errors, downstream timeouts, retries, idempotency, and error mapping.

10. How would you migrate a Mule flow to Node.js without breaking production?

Answer: I would migrate gradually, not as a big bang rewrite. First I would document the current Mule behavior, including mappings, retries, timeouts, error handling, and edge cases. Then I would build tests around the behavior, implement the Node version, compare outputs with shadow traffic where possible, and cut over incrementally with strong monitoring and rollback.

4. Data consistency and distributed systems

1. Why are distributed transactions usually not the best answer in an integration-heavy system?

Answer: Because they require all participants to support the same coordination model, they can block resources, and they become fragile across heterogeneous systems and external APIs. In an integration platform with internal services, databases, brokers, and third-party APIs, I would usually prefer local transactions, sagas, idempotency, and reconciliation instead of one global transaction.

2. How would you keep a booking flow consistent if it touches inventory, payment, booking, and loyalty systems?

Answer: I would model it as a saga. Each system performs a local transaction and the overall flow has explicit state. Critical steps like reserving inventory and authorizing payment need compensating actions, such as release seat or refund/void payment. Less critical steps, like loyalty points or notifications, can often be handled asynchronously and reconciled later if they fail.

3. What is a saga pattern?

Answer: A saga is a long-running business transaction split into multiple local transactions. Each step commits in its own system. If a later critical step fails, the saga runs compensating actions for the already completed steps. It is a practical way to handle consistency across services without a global distributed transaction.

4. Would you use an orchestrated or choreographed saga for a booking process?

Answer: For a booking process I would usually start with an orchestrated saga because the flow is business-critical, the order of steps matters, and debugging is important. A coordinator can store the saga state, call each step, and trigger compensation. Choreography can work well for looser cross-domain reactions, but the end-to-end flow is harder to see.

5. What problem does the outbox pattern solve?

Answer: It solves the problem where a service commits a database change but crashes before publishing the corresponding event. With the outbox pattern, the business change and the event record are written in the same local transaction. A separate publisher later sends the event to the broker. This prevents the database and the event stream from diverging silently.

6. If your message broker provides at-least-once delivery, what does that imply for consumers?

Answer: It means consumers may receive the same message more than once. Therefore the consumer must be idempotent. I can store processed event IDs, use unique constraints, use UPSERTs, or guard state transitions so that replaying the same event does not repeat the business side effect.

7. What does eventual consistency mean in practice?

Answer: It means different parts of the system may be temporarily out of sync, but they are designed to converge. For example, a booking may be confirmed before loyalty points appear. That is acceptable if the business accepts it, the UI communicates it clearly, and we monitor the lag so inconsistency does not last too long.

8. How would you handle a compensation that fails?

Answer: First, compensation itself should be idempotent and retryable. If it still fails, I would persist the saga state, mark it as needing retry or manual review, alert the team, and provide a runbook for recovery. The worst option is to lose the failure silently. In real systems, failed compensation must be visible and recoverable.

9. How would you prevent two users from reserving the last seat at the same time?

Answer: The inventory service should own that invariant and protect it at the data layer. Options include a database transaction with row-level locking, an atomic conditional update, optimistic locking with a version column, or a unique constraint depending on the model. I would not rely only on application-level checks because concurrent requests can pass the check at the same time.

10. How do you detect and recover from a saga stuck halfway?

Answer: I would store saga state durably with timestamps for each transition. A background process can find sagas that have not progressed within the expected time window and either retry, compensate, or escalate. I would also add structured logs, metrics for saga age and failure rate, tracing/correlation IDs, and an operational runbook.

5. Database topics

1. When would you choose Oracle over MySQL or PostgreSQL?

Answer: I would choose Oracle when the organisation already depends on Oracle-specific strengths: PL/SQL packages, stored procedures, mature enterprise tooling, vendor support, and long-lived core systems. For a greenfield relational project I might consider PostgreSQL as a strong default, but in an Oracle-heavy company the operational cost of moving away from Oracle can be higher than the technical benefit. Short version: I would choose Oracle when the organisation already depends on Oracle-specific strengths: PL/SQL packages, stored procedures, mature enterprise tooling, vendor support, and long-lived core systems.

2. What is the practical difference between Oracle and MySQL?

Answer: Both are relational databases and both use SQL, but they often appear in different environments. Oracle is common in enterprise systems with heavier stored procedure usage and mature operational tooling. MySQL is very common in web applications and app-centric stacks. I would not say one is globally better. I would choose based on workload, existing skills, licensing, tooling and consistency requirements. Short version: Both are relational databases and both use SQL, but they often appear in different environments.

3. What is a database transaction and why is it important?

Answer: A transaction groups multiple database operations into one unit of work. Either all changes are committed, or all are rolled back. This matters when partial updates would leave the system inconsistent, for example creating a booking, inserting passengers and updating payment status. Inside one database this is straightforward. Across several services or external systems, I would not assume a global transaction. I would think about compensation, retries, idempotency and status-based processing. Short version: A transaction groups multiple database operations into one unit of work.

4. What does ACID mean?

Answer: ACID means Atomicity, Consistency, Isolation and Durability. Atomicity means all parts succeed or all are rolled back. Consistency means the data remains valid according to rules and constraints. Isolation protects concurrent transactions from unsafe interference. Durability means committed data survives failures. In an interview I would connect

this to business data, because bookings, payments and inventory usually need strong correctness. Short version: ACID means Atomicity, Consistency, Isolation and Durability.

5. How would you investigate a slow database query?

Answer: I would first check whether the query is actually slow in the database or whether the application is waiting for a connection from the pool. Then I would look at query stats, execution plan, indexes, data volume, join pattern and lock waits. I would also check if the application is doing many small queries, such as an N+1 pattern. I would avoid blindly adding indexes before understanding the real bottleneck. Short version: I would first check whether the query is actually slow in the database or whether the application is waiting for a connection from the pool.

6. What is an index and when can it hurt performance?

Answer: An index helps the database find rows faster for certain access patterns, especially filters, joins, sorting and uniqueness checks. But indexes are not free. They use storage and slow down writes because inserts, updates and deletes must also update the index. So I would create indexes based on real query patterns and execution plans, not on every column by default. Short version: An index helps the database find rows faster for certain access patterns, especially filters, joins, sorting and uniqueness checks.

7. What is connection pooling and why does it matter in Node.js?

Answer: Opening a new database connection for every request is expensive and can overload the database. A connection pool keeps a managed set of open connections that the application reuses. The pool size must be tuned carefully

across all running instances. A too-small pool creates waiting, while a too-large pool can overload the database. I would also make sure every acquired connection is released in a finally block to prevent leaks. Short version: Opening a new database connection for every request is expensive and can overload the database.

8. Why do companies use stored procedures, and what are the downsides?

Answer: Stored procedures can encapsulate complex data operations behind a stable interface, reduce round trips, keep performance-sensitive logic close to the data and enforce rules consistently. The downside is that too much business logic in the database can be harder to version, test, debug and review. My balanced approach is to keep data-centric and performance-critical logic in stored procedures, but keep orchestration and API-level workflow in application code where possible. Short version: Stored procedures can encapsulate complex data operations behind a stable interface, reduce round trips, keep performance-sensitive logic close to the data and enforce rules consistently.

9. What are bind variables and why are they important in Oracle?

Answer: Bind variables are placeholders for values in SQL, such as WHERE id = :id. They are important for security because they prevent SQL injection, and they are important for performance because Oracle can reuse the parsed statement and execution plan. If values are concatenated into SQL strings, Oracle may see every value as a different statement, causing many hard parses. Bind variables make the statement stable and reusable. Short version: Bind variables are placeholders for values in SQL, such as WHERE id = :id.

10. What is the N+1 query problem and how would you fix it?

Answer: N+1 happens when the application loads a list with one query and then runs one more query for each item. For 100 items, that becomes 101 queries. It is common with ORMs because the generated SQL is hidden. I would fix it by checking query counts, using joins or controlled eager loading, batching queries, or writing explicit SQL for critical paths. The key is to understand the actual SQL, not just the ORM code. Short version: N+1 happens when the application loads a list with one query and then runs one more query for each item.

6. Integration patterns and Mule to Node.js migration

1. When would you choose synchronous communication and when would you choose asynchronous communication?

Answer: I would choose synchronous communication when the caller needs an immediate answer and the dependency chain is small and reliable enough. For example, if payment authorization must complete before confirming a booking, that step is probably synchronous. I would choose asynchronous communication when the work can happen later, when I need fan-out to multiple systems, when I want to absorb traffic spikes, or when I do not want one unavailable downstream system to block the original request. The key trade-off is simplicity and immediate feedback versus decoupling and resilience.

2. You need to notify five downstream systems when a booking is created. Would you do it synchronously or asynchronously?

Answer: Usually asynchronously. I would publish a BookingCreated event to a topic and let each downstream consumer process it independently with its own retry and dead-letter handling. That avoids coupling the booking API latency and availability to five other systems. The exception is a downstream system that is truly part of the critical path, such as payment authorization. That one may stay synchronous, while the other notifications move async.

3. One downstream service normally answers in 100 ms but starts taking 5 seconds. What can happen and how would you protect the system?

Answer: Without protection, requests start waiting, connection pools fill up, queues grow, latency spreads to endpoints that may not be directly related, and health checks may fail. That can become a cascading failure. I would protect the service with explicit per-downstream timeouts, limited retries with exponential backoff and jitter, a circuit breaker to fail fast while the dependency is unhealthy, and bulkheads or concurrency limits so one slow integration cannot consume all resources.

4. Explain orchestration versus choreography.

Answer: Orchestration means a central coordinator controls the workflow and calls each step. It is easier to understand and debug because the flow is explicit in one place. Choreography means services react to events and emit their own events without one central coordinator. It gives looser coupling and more independence, but the complete flow is harder to see without tracing. I would orchestrate inside a bounded workflow owned by one team, and use choreography across independent domains where services should react to events.

5. What is a dead letter queue and why is it important?

Answer: A dead letter queue is where messages go after they fail processing too many times. It prevents infinite retries and makes failures visible for investigation. A DLQ is only useful if it is monitored and if the team has a runbook for inspecting and safely reprocessing messages. In an integration system, DLQ handling is important because bad payloads, partner changes, or code bugs can otherwise create silent data loss or endless retry loops.

6. How would you implement retries for a third-party API call?

Answer: I would first decide whether the operation is safe to retry. For idempotent reads or operations protected by an idempotency key, retries are usually acceptable. I would use a small maximum retry count, exponential backoff, and jitter. I would not blindly retry validation errors or most 4xx responses. I would also set a total timeout budget, because retries must not make the caller wait forever. Finally, I would log attempts with correlation IDs so production issues are traceable.

7. What is a circuit breaker and how is it different from retry?

Answer: Retry tries the same operation again because the failure may be temporary. A circuit breaker stops calling a dependency for a while when it is clearly unhealthy. The purpose is to fail fast and protect the caller from wasting resources on calls that are very likely to fail. The common states are closed, open, and half-open. In practice I would often use retry and circuit breaker together, but with strict limits and monitoring.

8. What is a content-based router?

Answer: A content-based router sends a message to different handlers based on the message content. For example, a booking event might go to a different path depending on market, product type, or partner. In Node.js this could be implemented with a simple switch, a routing table, or a strategy pattern. The important part is to keep the routing rules explicit and testable, not hidden across many random if statements.

9. How would you translate a Mule DataWeave transformation into Node.js?

Answer: I would not just rewrite it line by line. First I would understand the input contract, the output contract, default values, error behavior, and edge cases. Then I would implement the transformation as explicit TypeScript mapping functions, with runtime validation at the boundaries. I would add tests using real examples from the Mule flow to make sure the Node implementation has the same behavior, including validation failures and unusual payloads.

10. What are the main risks when migrating integration flows from Mule to Node.js?

Answer: The main risk is losing hidden production behavior. Mule flows often contain retry settings, timeout values, transformation quirks, connector behavior, error mapping, and partner-specific edge cases learned over years. If we only rewrite the happy path, the new service may look cleaner but be less reliable. I would migrate incrementally, compare old and new behavior, add observability, use canary or shadow traffic where possible, and keep rollback options.

7. Node.js, Express and TypeScript

1. Why is Node.js a good fit for integration-heavy services?

Answer: Node.js is a good fit when the service is mostly I/O-bound. Integration services often spend most of their time waiting for external APIs, databases, queues, or message brokers. Node handles many concurrent I/O operations efficiently with its event-driven model. The important caveat is CPU-heavy work. I would avoid heavy synchronous processing on the main thread and offload it to worker threads, another service, or a managed compute option.

2. Can you explain the Node.js event loop in simple terms?

Answer: The event loop is the mechanism that keeps JavaScript work moving when async operations complete. Node does not block one thread per request while waiting for I/O. It starts the async operation, continues with other work, and resumes the callback or Promise continuation when the result is ready. The key production point is that heavy synchronous CPU work blocks the event loop and can make the whole service slow.

3. What is the difference between I/O-bound and CPU-bound work?

Answer: I/O-bound work spends most of its time waiting for external operations, such as HTTP calls, database queries, queues, or file operations. CPU-bound work spends most of its time calculating, for example encryption, image processing, large JSON transformation, or report generation. Node.js is usually strongest for I/O-bound services. CPU-heavy tasks should be offloaded or isolated so they do not block request handling.

4. How would you structure an Express service for production?

Answer: I would keep route handlers thin. Middleware would handle cross-cutting concerns such as request IDs, logging, authentication, authorization, validation, rate limiting, and error handling. Business logic would live in service or use-case modules. External systems would be accessed through clients or adapters. I would also standardize timeouts, structured logs, metrics, health checks, graceful shutdown, and consistent error responses.

5. What is Express middleware and why does order matter?

Answer: Middleware is code that runs during the request-response pipeline before the final response is sent. It can log, authenticate, validate, modify the request, or reject it. Order matters because earlier middleware affects later steps. For example, correlation ID middleware should run early so all logs include it. Validation should run before business logic. The central error middleware should run at the end so it can format failures consistently.

6. How should errors be handled in an Express API?

Answer: I would use central error handling instead of formatting errors differently in every route. Known domain errors should map to clear HTTP status codes, for example 404, 409, or 422. Dependency timeouts or unavailable downstream systems should be translated into controlled responses such as 503 or 504, depending on the case. Unexpected errors should be logged with context and request ID, but the API should not expose raw stack traces to consumers.

7. Why is TypeScript not enough for API input safety?

Answer: TypeScript checks code at compile time, but it disappears at runtime. Data coming from HTTP, queues, files, databases, or third-party APIs is still untrusted runtime data. So I would use runtime schema validation at system boundaries, for example with Zod, Joi, AJV, or JSON Schema. TypeScript and runtime validation work together: validation proves the shape at runtime, then TypeScript gives safer code after that point.

8. What TypeScript practices matter in a large backend codebase?

Answer: I would use strict mode where possible, avoid any in application code, prefer unknown for untrusted data until it is validated, and separate external DTOs from internal domain models. I would model states explicitly, for example with discriminated unions, and be careful with null and optional fields. The goal is not fancy types. The goal is to make contracts explicit and make invalid states harder to represent.

9. How would you handle CPU-intensive work in a Node.js integration service?

Answer: First I would avoid doing heavy synchronous work on the main thread. For medium CPU tasks I could use worker threads. For heavier or more specialized workloads I would move the work to a separate service or use a managed compute platform. For asynchronous processing I might put the work behind a queue so the API request path stays responsive. The main principle is to keep the event loop responsive.

10. What would you standardize in a Node.js integration platform after moving away from Mule?

Answer: I would standardize the things that Mule or an integration platform often gives you by default: HTTP client wrappers with timeouts, retries and circuit breaker behavior, structured logging with correlation IDs, boundary validation, consistent error mapping, typed configuration, health and readiness endpoints, graceful shutdown, telemetry, and a testing baseline. Without these standards, every service reinvents the same concerns and production behavior becomes inconsistent.

8. Performance and scaling

1. Can Node.js handle 100M requests per month?

Answer: Yes, in raw throughput terms that number is very reasonable. 100M per month is roughly 40 requests per second on average, and even a 5x peak is around 200 requests per second. The real challenge is not the average number. I would focus on p95 and p99 latency, slow dependencies, database pool saturation, retry storms, memory growth, and graceful degradation under load. With stateless services, horizontal scaling, good timeouts, observability, and proper dependency handling, Node.js can handle this kind of integration workload well.

2. An endpoint is slow in production. How would you investigate it?

Answer: I would first check scope and timing: is it one endpoint or the whole service, all users or one partner, and did it start after a deployment or traffic spike? Then I would look at metrics: latency percentiles, request rate, error rate, CPU, memory, database pool, downstream latency, and queue depth if relevant. After that I would use logs and traces with correlation IDs to see where the time is spent. I would avoid random optimization until I know whether the bottleneck is in our code, database, network, or a downstream system.

3. What is the difference between latency and throughput?

Answer: Latency is how long one request takes from the client perspective. Throughput is how many requests or messages the system can process per second or per minute. Both matter together. A system can have high throughput but bad latency if requests are queued for too long. In an API role, I would track average latency, but more importantly p95 and p99 latency, because tail latency often shows the real production pain.

4. How would you scale a stateless API service?

Answer: I would run multiple instances behind a load balancer and make sure any required state is externalized, for example in a database, cache, or token. The service should not depend on local memory for user or workflow state. I would also add proper readiness and liveness checks, graceful shutdown, environment-based configuration, and autoscaling based on meaningful signals such as CPU, request rate, latency, or queue lag. Horizontal scaling works best when the service is stateless and observable.

5. What can block the Node.js event loop?

Answer: Heavy synchronous CPU work can block the event loop. Examples include large JSON parsing, expensive loops, crypto, image processing, huge in-memory transformations, or synchronous file operations. When the event loop is blocked, unrelated requests can also become slow because JavaScript cannot continue processing callbacks and promise continuations. For CPU-heavy work I would use worker threads, a separate worker service, or an async queue, depending on the workload.

6. One downstream API becomes slow. How do you prevent it from taking down your service?

Answer: I would protect the call with a strict timeout, limited retries with exponential backoff and jitter, and a circuit breaker so we fail fast when the dependency is clearly unhealthy. I would also use bulkheads

or per-downstream concurrency limits so this dependency cannot consume all connections or request capacity. If the dependency is not on the critical path, I would consider degraded behavior, such as cached data, a default response, or a clean 503 with Retry-After.

7. What is backpressure and why does it matter?

Answer: Backpressure means the system has a way to slow down or reject work when incoming load is higher than safe processing capacity. Without it, the system may buffer too much, run out of memory, exhaust connection pools, or let every request time out deep in the stack. In HTTP, backpressure can mean rate limiting or returning 503 with Retry-After. In queues, it means monitoring lag, limiting producer speed, or scaling consumers carefully.

8. How does connection pooling affect performance?

Answer: Opening a database or HTTP connection for every request is expensive, so connection pools reuse existing connections. That improves performance and stability. But a larger pool is not always better. If the pool is too large, the service can overload the database or downstream system. If it is too small, requests wait for a free connection and latency increases. I would tune pools based on concurrency, database capacity, wait time, active connections, and error rates.

9. When would you use caching to improve performance?

Answer: I would cache data that is read often, relatively stable, and expensive to compute or fetch. For example, reference data, configuration-like data, or responses from a slow third-party API can be good candidates. I would be careful with data that changes frequently or has strong consistency requirements. The hard part is not only storing the value, but invalidation, TTL choice, cache stampede protection, and making sure stale data does not create business problems.

10. How would you load test an integration service?

Answer: I would test more than the happy path. First I would define expected average and peak traffic, target latency percentiles, and acceptable error rate. Then I would run tests for normal load, peak load, stress load, and longer soak tests to catch memory growth. Because this is an integration service, I would also simulate slow downstreams, timeouts, retries, and partial failures. During the test I would monitor CPU, memory, event loop lag, database pool, downstream latency, queue depth, and p95/p99 latency.

9. Production debugging and observability

1. How would you approach a production issue where users report intermittent failures?

Answer: I would first understand the scope and impact: which users, which endpoint, when it started, and whether it is intermittent or constant. Then I would check recent deployments or configuration changes. After that I would use metrics to see the error rate and latency pattern, traces to find where the request is failing or slowing down, and logs with the request ID to understand the detailed cause. I try to narrow the issue with evidence before changing anything.

2. What is the difference between logs, metrics, and traces?

Answer: Metrics show aggregate behavior over time, such as request rate, error rate, latency, queue depth, or memory usage. Traces show the path of one request across multiple services and help identify where time was spent. Logs give

detailed evidence inside one service or request. They are complementary: metrics show that there is a problem, traces show where it happens, and logs often explain why.

3. How would you debug a spike in 504 Gateway Timeout errors?

Answer: I would start with the dashboard and check which endpoints are affected and when the spike started. Then I would look for correlation with a deployment, config change, or dependency incident. I would inspect failing traces to find the slow span, especially outbound calls to databases or downstream services. If a downstream is slow, I would check timeout, retry, circuit breaker, and connection pool metrics. Finally, I would use the request ID to inspect detailed logs around the failing path.

4. What is a correlation ID and why is it important?

Answer: A correlation ID is an identifier attached to a request so logs from different services can be connected. At the edge of the system I either accept an incoming request ID or generate one. Then I propagate it to every downstream HTTP call and message, and include it in every log line. Without that convention, debugging a multi-service issue becomes a lot of manual timestamp matching and guesswork.

5. What is distributed tracing and what problem does it solve?

Answer: Distributed tracing records one request as it moves through multiple services. Each service creates spans with timing and status information. This solves the problem where every service has its own logs but no single view of the full request path. With tracing I can see, for example, that the total request took two seconds and most of that time was spent waiting for one downstream dependency.

6. What is the RED method?

Answer: RED means Rate, Errors, and Duration. For each service and ideally each endpoint, I want to know how much traffic is coming in, how many requests fail, and how long the requests take. It is a simple and practical way to build service dashboards and quickly detect production problems.

7. What is metric cardinality and why does it matter?

Answer: Cardinality is the number of unique label combinations created by a metric. Labels like method, endpoint, status code, and dependency are usually useful and bounded. Labels like user ID, request ID, or booking ID can create millions of time series and make the monitoring system slow or expensive. High-cardinality data usually belongs in logs or traces, not metric labels.

8. What would you instrument in an integration service?

Answer: At minimum I would instrument inbound request rate, error rate, and latency per endpoint. I would also instrument outbound calls per dependency: latency, error rate, timeout count, and retry count. For asynchronous processing I would track queue depth, message age, consumer errors, and dead letter queue size. For Node.js I would also monitor event loop lag, memory usage, CPU, and connection pool saturation.

9. How do you avoid logging sensitive data?

Answer: I would treat logs as long-lived operational data that may be copied to many systems. I would never log passwords, tokens, authorization headers, secrets, or unnecessary personal data. I would use allow-list logging for safe fields, mask sensitive values, and keep business identifiers only when they are needed for support or debugging. This is especially important in regulated or GDPR-sensitive environments.

10. If latency increases but error rate stays low, is that still a production problem?

Answer: Yes. Higher latency can be a real regression even before errors appear. I would compare p95 and p99 latency against the baseline and check whether the change correlates with a deployment or dependency slowdown. Then I would inspect traces to see where the extra time is spent. I would not continue a rollout just because the error count is still zero.

10. SOA, microservices, monolith and service boundaries

1. Luxair has many internal applications and third-party integrations. Would you recommend microservices?

Answer: Not automatically. I would first ask what problem we are trying to solve. If the main pain is integration fan-out, protocol transformation and central governance, a well-designed API layer or SOA-style integration layer may be better. If the pain is deployment coupling between teams and the domain boundaries are clear, then microservices can help. For Luxair I would probably start with coarse-grained Node services around clear capabilities and only split further where the operational benefit is real.

2. What is the difference between SOA and microservices?

Answer: Both use services, but the style is different. SOA often has a central integration layer or ESB that handles routing, transformation and orchestration. Services can be larger and sometimes share data models or infrastructure. Microservices usually push logic to the service edges, prefer independent deployment, stronger data ownership and lightweight communication. The trade-off is that microservices need stronger observability, CI/CD, contracts and operational discipline.

3. When would you still choose a monolith?

Answer: I would choose a monolith, or better a modular monolith, when the team is small, the domain is still changing, or the application is mostly an internal tool or simple business application. It keeps local transactions simple, debugging easier and operational cost lower. The important point is to keep internal module boundaries clean, so services can be extracted later if there is a real reason.

4. What makes a good service boundary?

Answer: A good service boundary follows a business capability, not a technical layer. The service should own logic and data that change for the same reasons. It should have a clear contract, low coupling to other services and high cohesion internally. If one simple business operation requires many chatty calls between services, the boundary is probably wrong.

5. How would you avoid distributed spaghetti in a microservice architecture?

Answer: I would avoid splitting too early and keep services aligned with business capabilities. I would define explicit API or event contracts, avoid shared databases, use correlation IDs and tracing, and keep synchronous chains short. I would also make ownership clear: each service should have a team that owns its code, data, monitoring and production behavior.

6. Should microservices share a database?

Answer: As a long-term design, I would avoid it because it creates hidden coupling. The database schema becomes an undocumented contract and one service can break another by changing tables. During migration a shared database may be a controlled temporary compromise, especially read-only access, but the target should be clear data ownership through APIs or events.

7. How do you decide between synchronous and asynchronous communication?

Answer: I use synchronous communication when the caller needs an immediate answer and the dependency is part of the critical path. I use asynchronous messaging when I need decoupling, fan-out, buffering or when the work can happen later. Async improves resilience and load handling, but it adds duplicate handling, retries, ordering questions and eventual consistency.

8. What is the role of an API gateway?

Answer: An API gateway is an entry layer in front of backend services. It can handle routing, authentication checks, rate limiting, logging, correlation IDs and sometimes protocol translation. It should simplify clients and protect services, but I would avoid putting too much business logic there, because then the gateway becomes a new hidden monolith.

9. Does migrating from Mule to Node.js mean moving to microservices?

Answer: No. Technology migration and architecture migration are separate decisions. We can replace Mule flows with Node.js services and still keep a coarse-grained SOA-style architecture. The key is to preserve behavior, contracts, error handling, retries, timeouts and monitoring. After that, we can decide case by case whether a flow should remain coarse-grained or be split into smaller services.

10. How would you know that splitting a service was worth it?

Answer: I would look for a concrete benefit: independent deployment, clearer ownership, separate scaling needs, reduced release coordination or better fault isolation. If the split only creates more network calls, more coordination and no team or business benefit, it is probably not worth it. A good split should reduce real pain, not just make the architecture look modern.